

Uni-App 页面布局完善与页面串联实现指南

页面布局的合理性与页面间跳转的流畅性是Uni-App项目用户体验的核心。本文将从项目整体结构规划入手，通过封装通用布局组件统一风格，结合路由配置与跳转逻辑实现各页面串联，同时融入适配技巧与交互优化，确保项目布局规范、跳转清晰。

一、前期准备：项目结构规划与路由配置

完善页面布局与串联前，需先明确项目结构与路由规则，为后续开发奠定基础。以典型电商项目为例，规划如下：

1. 规范项目目录结构

Code block

```
1  src/
2  |—— components/           // 通用组件（含布局组件）
3  |   |—— layout/           // 布局相关组件
4  |   |   |—— NavBar.vue    // 自定义导航栏
5  |   |   |—— TabBar.vue    // 自定义底部标签栏
6  |   |   |—— PageFrame.vue // 页面框架（统一包裹内容）
7  |   |—— common/           // 通用功能组件
8  |—— pages/                 // 业务页面
9  |   |—— index/            // 首页
10 |   |—— category/         // 分类页
11 |   |—— goods/            // 商品相关
12 |   |   |—— GoodsList.vue // 商品列表页
13 |   |   |—— GoodsDetail.vue // 商品详情页
14 |   |—— cart/             // 购物车页
15 |   |—— user/             // 个人中心页
16 |   |—— login/            // 登录页
17 |—— router/                // 路由配置（可选，Uni-App优先用pages.json）
18 |—— utils/                 // 公共方法
19 |—— static/                // 静态资源
20 |—— pages.json              // 全局配置（核心路由配置）
```

2. 配置pages.json实现路由基础串联

Uni-App通过pages.json配置页面路由与窗口样式，是页面串联的核心配置文件。需明确页面路径、窗口样式及底部标签栏（tabBar）配置，实现基础页面切换。

Code block

```
1  {
2    "pages": [
3      // 首页（默认启动页）
4      {
5        "path": "pages/index/index",
6        "style": {
7          "navigationStyle": "custom", // 关闭默认导航，使用自定义NavBar
8          "enablePullDownRefresh": true // 开启下拉刷新
9        }
10     },
11     // 分类页
12     {
13       "path": "pages/category/category",
14       "style": {
15         "navigationStyle": "custom"
16       }
17     },
18     // 商品列表页
19     {
20       "path": "pages/goods/GoodsList",
21       "style": {
22         "navigationStyle": "custom"
23       }
24     },
25     // 商品详情页
26     {
27       "path": "pages/goods/GoodsDetail",
28       "style": {
29         "navigationStyle": "custom"
30       }
31     },
32     // 购物车页
33     {
34       "path": "pages/cart/cart",
35       "style": {
36         "navigationStyle": "custom"
37       }
38     },
39     // 个人中心页
40     {
41       "path": "pages/user/user",
42       "style": {
43         "navigationStyle": "custom"
44       }
45     },
46     // 登录页
47     {
```

```

48     "path": "pages/login/login",
49     "style": {
50       "navigationStyle": "custom"
51     }
52   }
53 ],
54 // 底部标签栏配置（实现首页、分类、购物车、个人中心切换）
55 "tabBar": {
56   "color": "#999999", // 未选中颜色
57   "selectedColor": "#FF4444", // 选中颜色
58   "backgroundColor": "#FFFFFF",
59   "borderStyle": "black",
60   "list": [
61     {
62       "pagePath": "pages/index/index",
63       "text": "首页",
64       "iconPath": "static/tabbar/index.png",
65       "selectedIconPath": "static/tabbar/index-active.png"
66     },
67     {
68       "pagePath": "pages/category/category",
69       "text": "分类",
70       "iconPath": "static/tabbar/category.png",
71       "selectedIconPath": "static/tabbar/category-active.png"
72     },
73     {
74       "pagePath": "pages/cart/cart",
75       "text": "购物车",
76       "iconPath": "static/tabbar/cart.png",
77       "selectedIconPath": "static/tabbar/cart-active.png"
78     },
79     {
80       "pagePath": "pages/user/user",
81       "text": "我的",
82       "iconPath": "static/tabbar/user.png",
83       "selectedIconPath": "static/tabbar/user-active.png"
84     }
85   ]
86 },
87 "globalStyle": {
88   "backgroundColor": "#F5F5F5", // 页面背景色

```



tabBar配置的页面为“主页面”，相互切换时不会销毁页面；非tabBar页面为“二级页面”，跳转时会按路由栈管理，通过返回按钮或`uni.navigateBack`返回上一页。

二、核心：封装通用布局组件统一页面风格

为确保各页面布局统一、开发高效，需封装导航栏（NavBar）、页面框架（PageFrame）等通用布局组件，所有页面通过引入组件实现布局规范，同时预留页面个性化配置入口。

1. 封装自定义导航栏（NavBar.vue）

支持动态设置标题、左侧返回按钮（根据路由栈显示）、右侧功能按钮，适配不同页面需求。

Code block

```
1  <template>
2    <view class="nav-bar" :style="{ paddingTop: statusBarHeight + 'px' }">
3
4      <view class="nav-left" @click="handleBack" v-if="showBack">
5        <uni-icons type="back" size="28rpx" color="{{ navColor }}"></uni-icons>
6      </view>
7
8      <view class="nav-left placeholder" v-else></view>
9
10
11     <view class="nav-center">
12       <text class="nav-title" :style="{ color: navColor }">{{ navTitle }}
13     </text>
14   </view>
15
16   <view class="nav-right">
17     <slot name="right"></slot>
18   </view>
19 </view>
20 </template>
21
22 <script>
23   import utils from '@/utils/index.js';
24
25   export default {
26     props: {
27       navTitle: { // 导航栏标题
28         type: String,
29         default: '页面标题'
30       },
31       navColor: { // 导航栏文字颜色
32         type: String,
33         default: '#333333'
34       },
35     },
36   }
```

```
35     showBack: { // 是否显示返回按钮（默认根据路由栈判断）
36         type: Boolean,
37         default: null
38     }
39 },
40 data() {
41     return {
42         statusBarHeight: 0
43     };
44 },
45 onLoad() {
46     // 获取状态栏高度（从公共方法获取）
47     this.statusBarHeight = utils.getSystemInfo().statusBarHeight;
48 },
49 computed: {
50     // 自动判断是否显示返回按钮：路由栈长度>1时显示
51     isShowBack() {
52         if (this.showBack !== null) return this.showBack;
53         const pages = getCurrentPages();
54         return pages.length > 1;
55     }
56 },
57 methods: {
58     // 返回上一页
59     handleBack() {
60         const pages = getCurrentPages();
61         if (pages.length > 1) {
62             uni.navigateBack({ delta: 1 });
63         } else {
64             // 路由栈只有一页时，返回首页（避免无法返回）
65             uni.switchTab({ url: '/pages/index/index' });
66         }
67     }
68 }
69 }
70 </script>
71
72 <style scoped lang="scss">
73 .nav-bar {
74     width: 100%;
75     background-color: #FFFFFF;
76     box-shadow: 0 2rpx 10rpx rgba(0, 0, 0, 0.05);
77 }
78
79 .nav-left {
80     position: absolute;
81     left: 20rpx;
```

```
82     top: 0;
83     height: 80rpx;
84     display: flex;
85     align-items: center;
86     width: 60rpx;
87 }
```

2. 封装页面框架（PageFrame.vue）

统一包裹页面内容，集成导航栏、下拉刷新、上拉加载等通用功能，减少页面重复代码。

Code block

```
1  <template>
2    <view class="page-frame">
3
4      <nav-bar
5        :nav-title="pageTitle"
6        :nav-color="navColor"
7        :show-back="showBack"
8      >
9        <slot name="nav-right"></slot>
10    </nav-bar>
11
12
13    <scroll-view
14      class="page-content"
15      scroll-y
16      :enable-pull-down-refresh="enablePullDown"
17      :onPullDownRefresh="handlePullDown"
18      :onReachBottom="handleReachBottom"
19      :lower-threshold="100"
20    >
21      <slot></slot>
22
23
24      <view class="loading-more" v-if="isLoadingMore">
25        <uni-loading type="circle" size="24rpx"></uni-loading>
26        <text class="loading-text">加载中...</text>
27      </view>
28    </scroll-view>
29  </view>
30 </template>
31
32 <script>
33 import NavBar from './NavBar.vue';
34 import uniLoading from '@components/uni-ui/uni-loading/uni-loading.vue';
35
```

```
36 export default {
37   components: { NavBar, uniLoading },
38   props: {
39     pageTitle: { // 页面标题（传递给导航栏）
40       type: String,
41       default: '页面标题'
42     },
43     navColor: { // 导航栏文字颜色
44       type: String,
45       default: '#333333'
46     },
47     showBack: { // 是否显示返回按钮
48       type: Boolean,
49       default: null
50     },
51     enablePullDown: { // 是否开启下拉刷新
52       type: Boolean,
53       default: false
54     },
55     isLoadingMore: { // 是否显示上拉加载提示
56       type: Boolean,
57       default: false
58     }
59   },
60   methods: {
61     // 下拉刷新回调（传递给父页面）
62     handlePullDown() {
63       this.$emit('pull-down');
64     },
65     // 上拉加载回调（传递给父页面）
66     handleReachBottom() {
67       this.$emit('reach-bottom');
68     }
69   }
70 }
71 </script>
72
73 <style scoped lang="scss">
74 .page-content {
75   min-height: calc(100vh - var(--status-bar-height, 0px) - 80rpx);
76   padding-bottom: 20rpx;
77   box-sizing: border-box;
78 }
79
80 .loading-more {
81   display: flex;
82   align-items: center;
```

```
83     justify-content: center;
84     height: 80rpx;
85     font-size: 24rpx;
86     color: #999999;
87   }
88
```

三、页面布局实现：典型页面示例与串联逻辑

基于通用布局组件，实现首页、商品列表、商品详情等典型页面的布局，同时通过路由跳转实现页面串联，明确不同场景下的跳转方式（如tab切换、普通跳转、带参数跳转）。

1. 首页布局（index.vue）：tab主页面，跳转至商品列表/详情

首页作为入口，包含轮播图、分类入口、推荐商品列表，点击商品跳转至商品详情页，点击分类入口跳转至分类页。

Code block

```
1  <template>
2    <page-frame
3      page-title="首页"
4      :enable-pull-down="true"
5      @pull-down="onPullDownRefresh"
6    >
7
8    <uni-swiper
9      class="home-swiper"
10     :autoplay="true"
11     :interval="3000"
12     :duration="500"
13   >
14     <uni-swiper-item v-for="(item, index) in swiperList" :key="index">
15       <image :src="item.image" mode="widthFix" class="swiper-img"></image>
16     </uni-swiper-item>
17   </uni-swiper>
18
19
20   <view class="category-list">
21     <view class="category-item" v-for="(item, index) in categoryList"
22       :key="index" @click="toCategory(item.id)">
23       <image :src="item.icon" mode="widthFix" class="category-icon"></image>
24       <text class="category-name">{{ item.name }}</text>
25     </view>
26   </view>
27
28   <view class="goods-list">
```



```
29     <view class="list-title">
30         <text class="title-text">推荐商品</text>
31         <text class="more-text" @click="toGoodsList">查看更多 ></text>
32     </view>
33
34     <view class="goods-grid">
35         <view class="goods-item" v-for="(item, index) in recommendGoods"
:   key="index" @click="toGoodsDetail(item.id)">
36             <image :src="item.cover" mode="widthFix" class="goods-img"></image>
37             <text class="goods-name">{{ item.name }}</text>
38             <text class="goods-price">¥{{ item.price.toFixed(2) }}</text>
39         </view>
40     </view>
41 </page-frame>
42 </template>
43
44 <script>
45 import PageFrame from '@/components/layout/PageFrame.vue';
46 import uniSwiper from '@/components/uni-ui/uni-swiper/uni-swiper.vue';
47 import uniSwiperItem from '@/components/uni-ui/uni-swiper/uni-swiper-
item.vue';
48 import utils from '@/utils/index.js';
49
50 export default {
51     components: { PageFrame, uniSwiper, uniSwiperItem },
52     data() {
53         return {
54             swiperList: [ // 轮播图数据
55                 { image: '/static/swiper/1.png' },
56                 { image: '/static/swiper/2.png' }
57             ],
58             categoryList: [ // 分类数据
59                 { id: 1, name: '服饰', icon: '/static/category/1.png' },
60                 { id: 2, name: '数码', icon: '/static/category/2.png' }
61             ],
62             recommendGoods: [] // 推荐商品数据
63         };
64     },
65     onLoad() {
66         this.getRecommendGoods();
67     },
68     methods: {
69         // 获取推荐商品数据
70         getRecommendGoods() {
71             // 模拟接口请求
72             uni.request({
73                 url: 'https://api.example.com/goods/recommend',
```

```

74         success: (res) => {
75             this.recommendGoods = res.data.data;
76         }
77     });
78 },
79 // 下拉刷新
80 onPullDownRefresh() {
81     this.getRecommendGoods();
82     // 停止下拉刷新动画
83     setTimeout(() => {
84         uni.stopPullDownRefresh();
85     }, 500);
86 }

```

2. 商品详情页 (GoodsDetail.vue)：二级页面，跳转至购物车/返回列表

接收首页传递的goodsId参数，请求商品详情数据，包含商品信息、加入购物车按钮，点击返回按钮返回上一页，点击购物车按钮跳转至购物车tab页。

Code block

```

1  <template>
2    <page-frame
3      :page-title="goodsInfo.name || '商品详情'"
4      nav-color="#FFFFFF"
5      :show-back="true"
6    >
7
8    <template #nav-right>
9      <uni-icons
10        type="star"
11        size="28rpx"
12        color="#FFFFFF"
13        @click="handleCollect"
14        class="collect-icon"
15      ></uni-icons>
16    </template>
17
18
19    <image :src="goodsInfo.cover" mode="widthFix" class="goods-big-img">
20    </image>
21
22    <view class="goods-info">
23      <text class="goods-name">{{ goodsInfo.name }}</text>
24      <text class="goods-price">¥{{ goodsInfo.price.toFixed(2) }}</text>
25      <text class="goods-desc">{{ goodsInfo.desc }}</text>
26    </view>

```

```
27
28
29 <view class="goods-attr">
30   <view class="attr-item" v-for="(item, key) in goodsInfo.attr" :key="key">
31     <text class="attr-key">{{ key }}: </text>
32     <text class="attr-value">{{ item }}</text>
33   </view>
34 </view>
35
36
37 <view class="goods-operation">
38   <view class="op-item" @click="toCart">
39     <uni-icons type="cart" size="32rpx" color="#FFFFFF"></uni-icons>
40     <text class="op-text">购物车</text>
41   </view>
42   <view class="op-item buy-now" @click="buyNow">
43     <text class="op-text">立即购买</text>
44   </view>
45   <view class="op-item add-cart" @click="addToCart">
46     <text class="op-text">加入购物车</text>
47   </view>
48 </view>
49 </page-frame>
50 </template>
51
52 <script>
53 import PageFrame from '@/components/layout/PageFrame.vue';
54 import uniIcons from '@/components/uni-ui/uni-icons/uni-icons.vue';
55 import utils from '@/utils/index.js';
56
57 export default {
58   components: { PageFrame, uniIcons },
59   data() {
60     return {
61       goodsId: '',
62       goodsInfo: {} // 商品详情数据
63     };
64   },
65   onLoad(options) {
66     // 接收上一页传递的goodsId参数
67     this.goodsId = options.goodsId;
68     if (this.goodsId) {
69       this.getGoodsDetail();
70     } else {
71       uni.showToast({ title: '商品ID异常', icon: 'none' });
72       setTimeout(() => {
73         uni.navigateBack({ delta: 1 });
```

```

74     }, 1000);
75   }
76 },
77 methods: {
78   // 获取商品详情数据
79   getGoodsDetail() {
80     uni.showLoading({ title: '加载中' });
81     uni.request({
82       url: `https://api.example.com/goods/detail?id=${this.goodsId}`,
83       success: (res) => {
84         this.goodsInfo = res.data.data;
85       },
86       fail: () => {
87         uni.showToast({ title: '获取商品信息失败', icon: 'none' });

```

3. 购物车页（cart.vue）：tab主页面，关联商品详情与结算页

展示用户加入购物车的商品，点击商品跳转至商品详情页，点击结算按钮跳转至订单确认页。

Code block

```

1  <template>
2    <page-frame
3      page-title="购物车"
4      :enable-pull-down="true"
5      @pull-down="onPullDownRefresh"
6    >
7
8    <view class="empty-cart" v-if="cartList.length === 0">
9      <image src="/static/empty-cart.png" mode="widthFix" class="empty-img">
10     </image>
11     <text class="empty-text">购物车是空的哦~</text>
12     <button class="go-shopping" @click="toHome">去购物</button>
13   </view>
14
15   <view class="cart-list" v-else>
16     <view class="cart-item" v-for="(item, index) in cartList" :key="index">
17       <uni-checkbox v-model="item.checked" class="cart-checkbox"></uni-
18       checkbox>
19       <image :src="item.cover" mode="widthFix" class="cart-img"></image>
20       <view class="cart-info">
21         <text class="cart-name" @click="toGoodsDetail(item.goodsId)">{{
22         item.name }}</text>
23         <text class="cart-price">¥{{ item.price.toFixed(2) }}</text>
24         <view class="cart-num">
25           <uni-icons type="minus" size="24rpx" @click="changeNum(index, -1)">
26         </view>
27       </view>
28     </view>
29   </view>
30 </template>

```

```
24         <text class="num-text">{{ item.num }}</text>
25         <uni-icons type="plus" size="24rpx" @click="changeNum(index, 1)">
</uni-icons>
26     </view>
27 </view>
28     <uni-icons type="trash" size="24rpx" color="#999999"
@click="deleteCart(index)" class="cart-delete"></uni-icons>
29 </view>
30 </view>
31
32
33     <view class="cart-footer" v-if="cartList.length > 0">
34         <uni-checkbox v-model="selectAll" class="footer-checkbox"
@click="handleSelectAll">
35             <text class="select-all-text">全选</text>
36         </uni-checkbox>
37         <view class="footer-total">
38             <text class="total-text">合计: ¥{{ totalPrice.toFixed(2) }}</text>
39             <text class="total-desc">共{{ selectedNum }}件商品</text>
40         </view>
41         <button class="footer-pay" @click="toConfirmOrder">结算</button>
42     </view>
43 </page-frame>
44 </template>
45
46 <script>
47 import PageFrame from '@/components/layout/PageFrame.vue';
48 import uniCheckbox from '@/components/uni-ui/uni-checkbox/uni-checkbox.vue';
49 import uniIcons from '@/components/uni-ui/uni-icons/uni-icons.vue';
50
51 export default {
52     components: { PageFrame, uniCheckbox, uniIcons },
53     data() {
54         return {
55             cartList: [], // 购物车数据
56             selectAll: false // 全选状态
57         };
58     },
59     onShow() {
60         // 页面显示时获取购物车数据
61         this.getCartList();
62     },
63     computed: {
64         // 选中商品总数
65         selectedNum() {
66             return this.cartList.filter(item => item.checked).reduce((total, item)
=> total + item.num, 0);
```

```
67     },
68     // 选中商品总价
69     totalPrice() {
70         return this.cartList.filter(item => item.checked).reduce((total, item)
=> total + item.price * item.num, 0);
71     }
72 },
73 methods: {
74     // 获取购物车数据
75     getCartList() {
76         uni.request({
77             url: 'https://api.example.com/cart/list',
78             success: (res) => {
79                 this.cartList = res.data.data.map(item => ({ ...item, checked: false
```

四、页面串联核心：路由跳转方式与参数传递

Uni-App提供多种路由跳转API，需根据页面类型（tab页/非tab页）选择合适的跳转方式，同时掌握参数传递与接收的方法，确保页面间数据互通。

1. 路由跳转方式对比

API	适用场景	特点	示例
uni.switchTab	tabBar页面之间的切换	关闭所有非tab页，跳转到指定tab页，不支持传递参数	跳转到首页、购物
uni.navigateTo	非tab页跳转（二级页面）	保留当前页面，跳转到新页面，可通过URL传递参数，有返回按钮	首页→商品详情、算页
uni.navigateBack	二级页面返回上一页	关闭当前页面，返回上一页面或多级页面，可通过delta控制返回层级	商品详情→首页、物车
uni.redirectTo	非tab页跳转（替换当前页面）	关闭当前页面，跳转到新页面，无返回按钮，可传递参数	登录页→个人中心
uni.reLaunch	任意页面跳转（重置路由栈）	关闭所有页面，跳转到新页面，可跳转到tab页或非tab页	退出登录后跳转到

2. 参数传递与接收方法

(1) URL参数传递（适用于简单数据）

通过URL拼接参数，适用于字符串、数字等简单数据，参数长度有限制（约1024字节）。

Code block

```
1  // 跳转页：传递参数 (goodsId和name)
2  uni.navigateTo({
3    url: `/pages/goods/GoodsDetail?goodsId=${123}&name=${encodeURIComponent('商品名称')}`
4  });
5
6  // 接收页：onLoad生命周期中接收
7  onLoad(options) {
8    const goodsId = options.goodsId; // 123
9    const name = decodeURIComponent(options.name); // 商品名称
10 }
```

(2) 全局变量传递（适用于复杂数据）

通过`Vue.prototype`或`getApp()`设置全局变量，适用于对象、数组等复杂数据，页面销毁后仍可访问。

Code block

```
1  // main.js中定义全局变量
2  Vue.prototype.$globalData = {
3    currentCategory: {}
4  };
5
6  // 跳转页：设置全局变量
7  this.$globalData.currentCategory = { id: 1, name: '服饰' };
8  uni.switchTab({ url: '/pages/category/category' });
9
10 // 接收页：获取全局变量
11 onShow() {
12   const currentCategory = this.$globalData.currentCategory;
13   console.log(currentCategory); // { id: 1, name: '服饰' }
14 }
```

(3) Vuex传递（适用于跨组件/页面数据共享）

对于需要在多个页面共享的数据（如用户信息、购物车数据），推荐使用Vuex存储与传递。

Code block

```
1  // store/index.js
2  import Vue from 'vue';
3  import Vuex from 'vuex';
```

```

4
5  Vue.use(Vuex);
6
7  export default new Vuex.Store({
8    state: {
9      userInfo: null
10   },
11   mutations: {
12     setUserInfo(state, data) {
13       state.userInfo = data;
14     }
15   },
16   actions: {
17     login({ commit }, data) {
18       // 模拟登录接口
19       commit('setUserInfo', data);
20     }
21   },
22   getters: {
23     isLogin(state) {
24       return !!state.userInfo;
25     }
26   }
27 });
28
29 // 登录页：提交Vuex
30 this.$store.dispatch('login', { id: 1, name: '用户' });
31
32 // 个人中心页：获取Vuex数据
33 computed: {
34   ...mapGetters(['isLogin']),
35   userInfo() {
36     return this.$store.state.userInfo;
37   }
38 }

```

五、优化与适配：提升页面体验的关键技巧

1. 布局适配：适配不同屏幕尺寸

- 使用rpx作为尺寸单位，Uni-App会自动将rpx转换为对应设备的px（750rpx=屏幕宽度）；
- 通过`uni.getSystemInfoSync()`获取屏幕尺寸，动态调整布局（如宽高比）；
- 避免使用固定px单位，特殊场景（如状态栏高度）需用px时，结合条件编译适配不同平台。

2. 交互优化：提升页面跳转流畅性

- 跳转前加载数据：对于需要请求数据的页面，可在跳转前预加载部分数据，减少页面加载等待时间；
- 添加过渡动画：通过`pages.json`配置页面跳转动画，提升体验：

```
"style": {  
  "app-plus": {  
    "animationType": "slide-in-right", // 右侧滑入  
    "animationDuration": 300  
  }  
}
```

- 返回时携带数据：通过`getCurrentPages()`获取上一页实例，传递返回数据：

```
<pre
```

（注：文档部分内容可能由 AI 生成）